



CHAPTER 4

HARDWARE AND SOFTWARE IMPLEMENTATION

This chapter describes the implementation of the Smart Attendance System as a connected hardware-software platform. The classroom unit uses an ESP32-S3 development board, an OV2640 camera, a buzzer, green and red status LEDs, a PCB board, and a protective junction box. The unit communicates with a responsive Angular portal, a Spring Boot application programming interface, PostgreSQL persistence, and an independent FastAPI face-recognition service. Together, these elements support automatic session discovery, face-based attendance, weighted roll calls, dashboards, and university-style reports.

4.1. Methodology

The project followed an incremental implementation methodology. The first stage established users, roles, departments, student and teacher profiles, secure login, and token renewal. The second stage added courses, enrollments, timetables, and controlled attendance-session transitions. The third stage integrated face registration and verification with the ESP32-S3 classroom device and the student's mobile camera. The final stage implemented dashboards, audit logs, system settings, study-year reports, downloadable PDF and Excel files, and responsive portal layouts.

Each increment was implemented as an end-to-end vertical slice. A database migration defined the persistent model, the Spring service enforced business rules, a REST controller exposed the contract, and the Angular service and component presented the operation to the correct role. Automated integration tests were used to confirm authentication, authorization, academic consistency, face-service communication, device behavior, report generation, and dashboard calculations.

The methodology emphasized server-side authority. The backend resolves the student's study year, active session, eligible candidate list, roll-call count, and attendance totals instead of trusting the browser or classroom controller. This keeps the ESP32-S3 firmware focused on capture and feedback while ensuring that web and hardware attendance follow the same academic rules.

4.1.1. Block Diagram of the System

Administrators, teachers, and students use one responsive Angular portal. The portal sends HTTPS requests to the Spring Boot API, which applies identity, role, enrollment, timetable, and duplicate-attendance rules. PostgreSQL stores academic and attendance records. FastAPI performs face embedding operations. The ESP32-S3 unit acts as the authenticated classroom client and combines the OV2640 camera with visual and audible feedback components.

Smart Attendance System Architecture

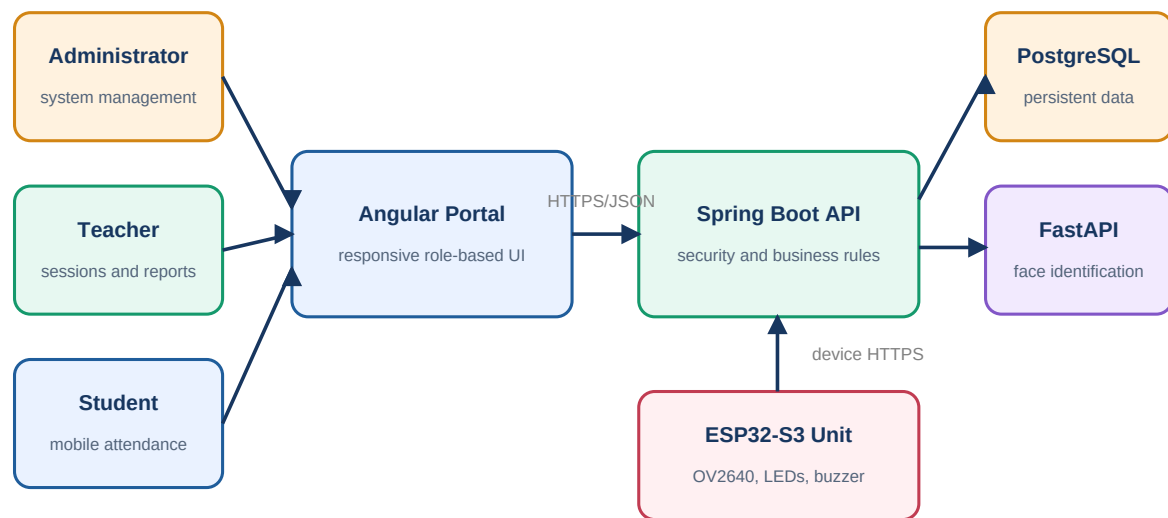


Figure 4.1 Block diagram of the Smart Attendance System

The Angular portal and ESP32-S3 unit do not access PostgreSQL or the recognition store directly. Spring Boot obtains academic values from the authenticated device binding and active timetable context. FastAPI receives only candidate students who are eligible for the resolved course, which narrows the identification search and isolates unrelated biometric records.

4.1.2. System Flow Chart

The operational flow begins with a recurring timetable. A scheduler creates the day's attendance session, starts it at the class start time, and closes it at the end time. During an active session, the ESP32-S3 with OV2640 camera or an authenticated student's mobile camera captures a face image. Spring validates the request before calling FastAPI. A successful match is inserted once and contributes the configured number of roll calls to reports.

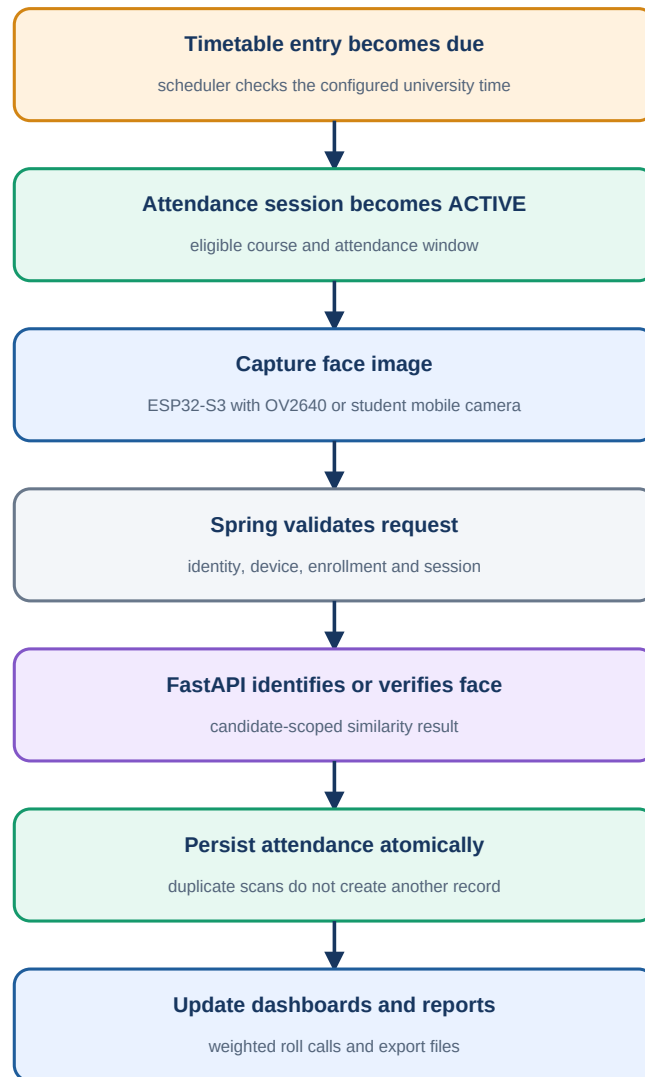


Figure 4.2 Flow chart for automated attendance processing

If there is no active session, the device binding is ambiguous, the student is not enrolled, recognition does not meet the configured threshold, or attendance already exists, the system returns a controlled result without creating another attendance row.

4.1.3. Data and Security Design

The data model separates application users from student and teacher profiles. Departments group users and courses; enrollments connect students to courses; timetable entries define recurring class periods; sessions represent dated classes; and attendance records connect verified students to those sessions. A roll-call count from one to six represents consecutive attendance units within a class period.

Control	Implementation	Purpose
User authentication	Signed JWT access tokens and rotating refresh tokens	Protect browser requests and support session renewal
Role authorization	ADMIN, TEACHER, and STUDENT authorities	Restrict portal and API operations by responsibility
Password storage	BCrypt hashes	Prevent plaintext password storage
Device authentication	Device identifier and BCrypt-hashed device secret	Authenticate the ESP32-S3 classroom unit
Biometric isolation	Spring stores only an opaque embedding identifier	Keep face embeddings in the independent FastAPI service
Input validation	JPEG, PNG, or WebP validation with a 10 MB limit	Reject empty, unsupported, or oversized images
Auditability	Append-only audit records	Record login, attendance, updates, deletes, and downloads

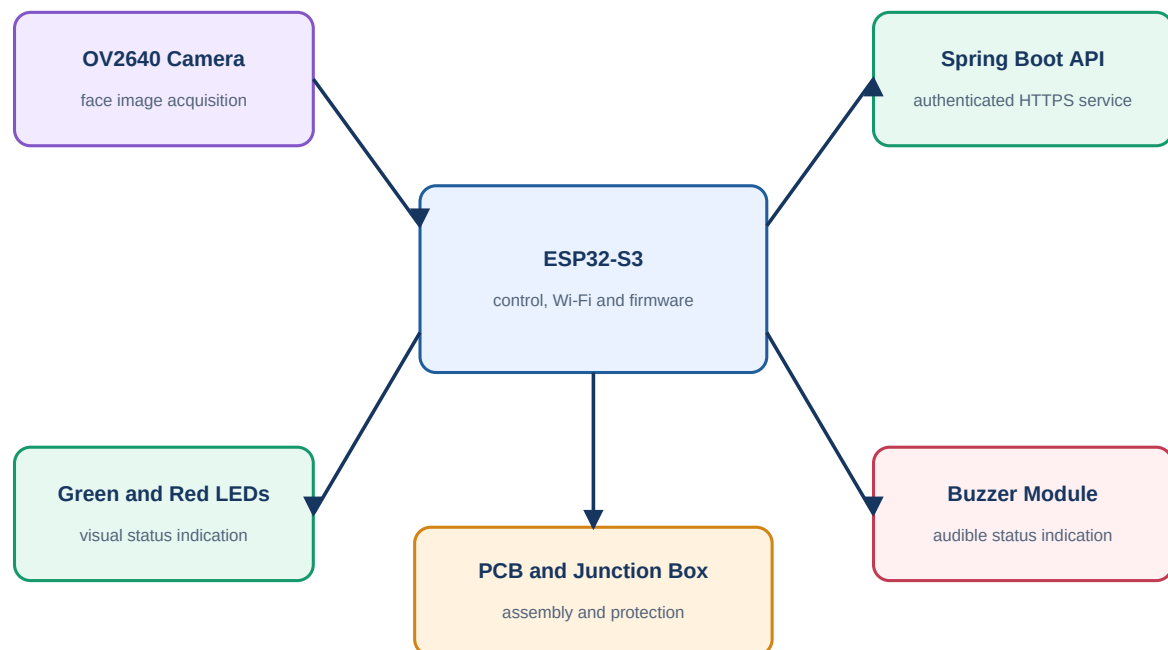
Table 4.1 Principal security and data-integrity controls

Database uniqueness constraints and service checks prevent duplicate enrollment, duplicated generated sessions, and duplicated student attendance. Teacher access is restricted to assigned courses, and student verification is restricted to the authenticated student's own attendance.

4.2. Hardware Implementation

The physical classroom unit is built around an ESP32-S3 development board. An OV2640 camera module provides image capture, while Wi-Fi on the controller supports communication with the backend service. A buzzer provides audible feedback, and green and red LEDs provide immediate visual status. The components are mounted on a PCB board and placed inside a junction box to organize connections and protect the assembled electronics.

ESP32-S3 Attendance Hardware Integration



Exact electrical connections follow the final circuit diagram and assembled PCB.

Figure 4.3 Integration of the actual Smart Attendance hardware

Hardware item	Implementation role
ESP32-S3 development board	Runs the device firmware, controls peripherals, and communicates through Wi-Fi
OV2640 camera	Captures the face image submitted for identification
Buzzer module	Provides an audible response for device or attendance status
Green and red LEDs	Provide visible success, rejection, or device-status indication
PCB board	Supports permanent component mounting and organized interconnections
Junction box	Encloses and protects the assembled classroom device

Table 4.2 Hardware components used in the prototype

4.2.1. Hardware Components and Assembly

Figure 4.4 presents the controller and image-capture components selected for the prototype. The ESP32-S3 provides the processing and network interface. The OV2640 connects through the camera interface so that the firmware can acquire a compressed face image before transmission.

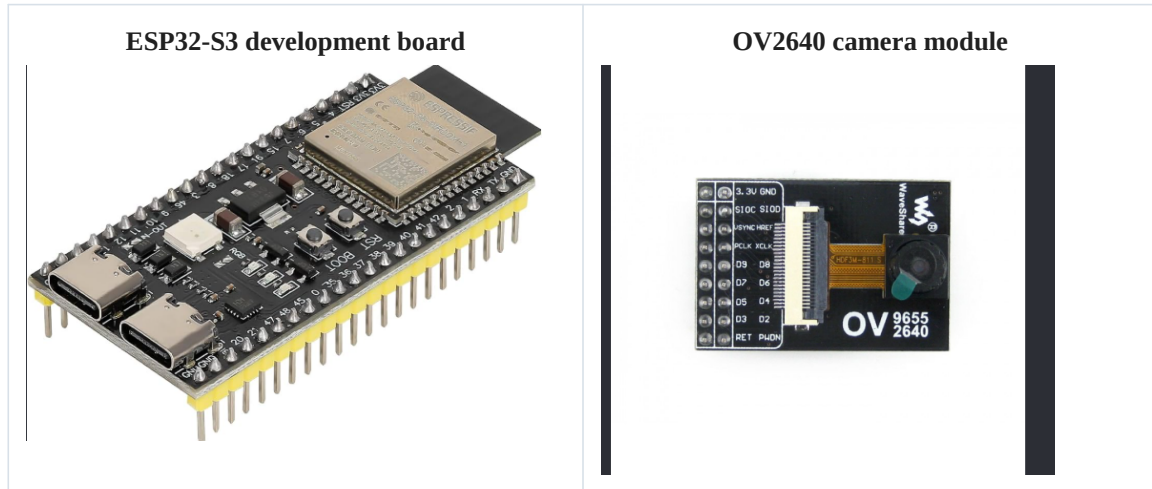


Figure 4.4 ESP32-S3 controller and OV2640 camera

The feedback and packaging components are shown in Figure 4.5. The buzzer and the green and red LEDs communicate device status locally. The PCB board supports a stable assembled circuit, and the junction box provides an enclosure suitable for mounting the camera and indicators. The final circuit diagram and bill of materials should record the exact GPIO assignments, current-limiting resistors, buzzer interface, and regulated supply used in the assembled unit.

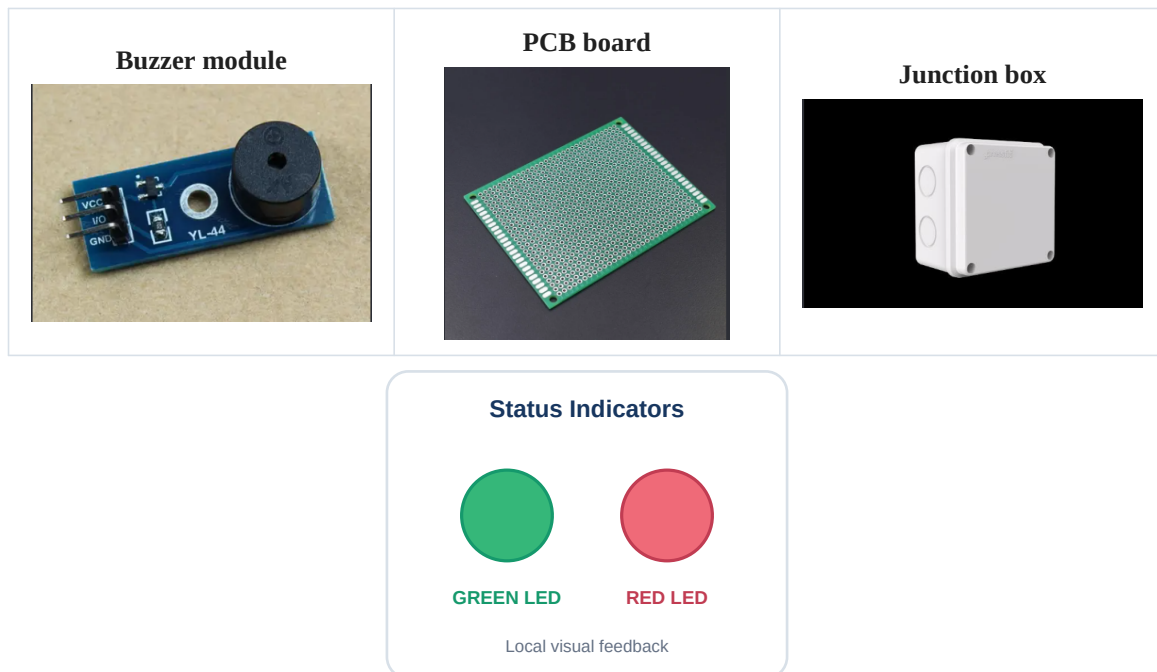


Figure 4.5 Buzzer, PCB board, junction box, and status LEDs

4.2.2. ESP32-S3 Attendance Operation

Before installation, an administrator registers the classroom unit through the hardware-management screen and binds it to a room, course, or both. The device identifier and secret are provisioned in the ESP32-S3 firmware together with the Wi-Fi connection. The backend stores only a BCrypt hash of the device secret.

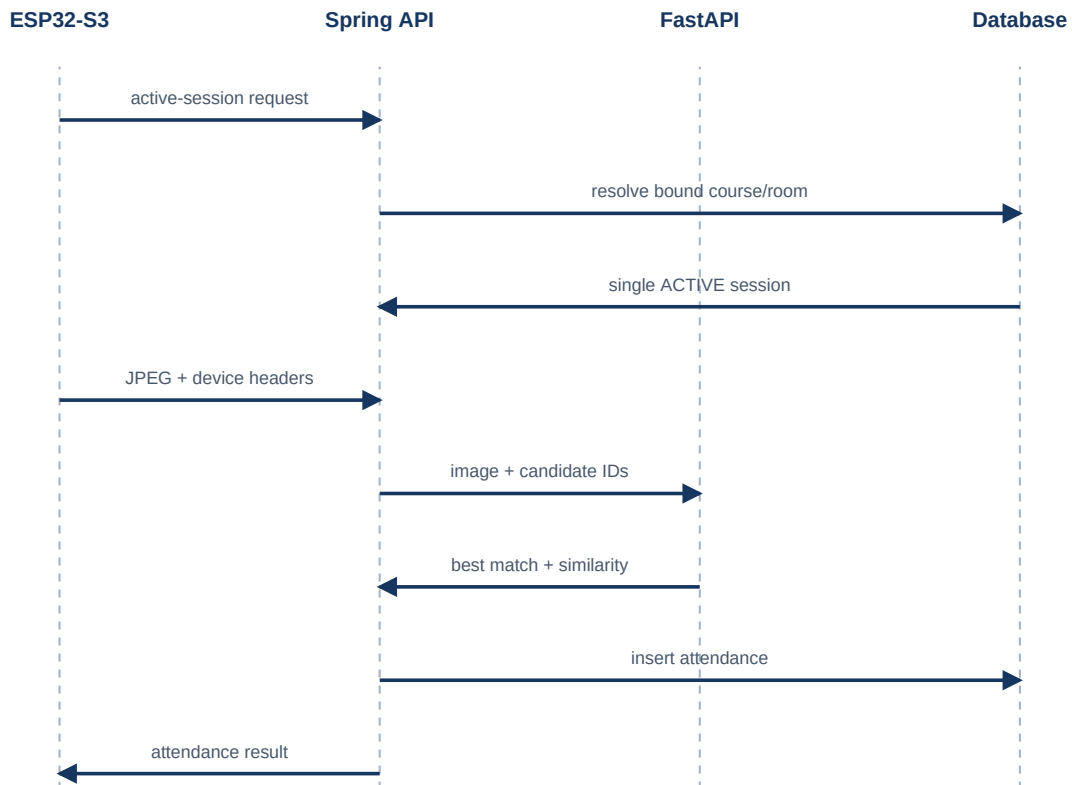


Figure 4.6 Message sequence for automatic device attendance

The device requests the active session associated with its binding, captures a JPEG through the OV2640, and submits the image with its authentication headers. Spring limits FastAPI identification to enrolled students with registered faces, validates the returned result, inserts attendance, and sends a controlled response to the device. The firmware then uses the LEDs and buzzer to provide local feedback. Repeated scans return an `ALREADY_VERIFIED` result without inserting a second record.

Session identifiers and academic values remain on the server, so timetable changes do not require a new hard-coded session identifier in the firmware. This keeps the classroom device reusable and preserves one authoritative source for academic policy.

4.3. Software Implementation

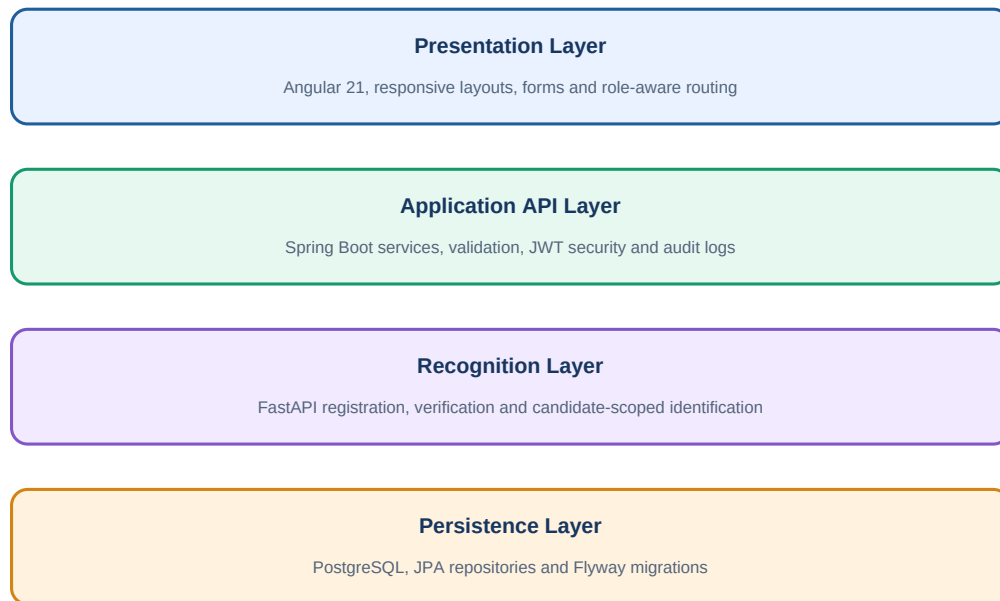


Figure 4.7 Layered software implementation

4.3.1. Angular Web Portal

The frontend uses Angular 21, TypeScript, reactive forms, RxJS, lazy-loaded routes, and standalone components. A shared HTTP interceptor attaches access tokens and performs refresh-token rotation. Route guards protect authenticated areas and direct users to the correct administrator, teacher, or student portal.

Administrators manage users, departments, courses, enrollments, assignments, timetables, sessions, hardware devices, reports, settings, and audit logs. Teachers manage assigned courses, attendance sessions, records, study-year cohort percentages, and Excel downloads. Students view their dashboard, timetable, courses, attendance history, face registration, and mobile camera verification.

Responsive breakpoints replace wide attendance tables with mobile cards so attendance percentages remain visible on small screens. Forms provide validation messages, loading indicators, retry actions, empty states, and protected one-time display of device secrets.

4.3.2. Spring Boot Application

The backend uses Java 21 and Spring Boot. Spring Security validates JWTs and role authorities. Spring Data JPA maps application entities to PostgreSQL, Flyway applies versioned migrations, Bean Validation checks requests, and centralized exception handling returns consistent JSON errors.

Business services enforce ownership and academic consistency rather than relying on controller checks. Hardware units are resolved from database-backed bindings. Automatic timetable processing uses uniqueness rules so application restarts do not create duplicate sessions.

4.3.3. Face Recognition and Image Processing

Face processing is separated from the academic system. During registration, Spring validates the uploaded image and sends it to FastAPI, then stores only the returned embedding identifier. During student verification, Spring sends the image and expected identity. During ESP32-S3 identification, it sends the image and only the identifiers of eligible candidate students. FastAPI detects the face, evaluates liveness when available, generates an embedding, compares eligible embeddings, and returns the best result with a similarity value.

This separation allows the recognition implementation to evolve independently while Spring remains responsible for enrollment, session state, duplicate prevention, and reporting. FastAPI does not require direct access to the academic PostgreSQL tables.

4.3.4. Attendance Calculation and Reporting

Attendance reports use weighted roll calls. For each eligible session, the session's roll-call count contributes to the denominator. A verified student receives that number of present units; absence is derived as eligible units minus present units. Course percentages use weighted totals, while overall attendance is averaged across the courses in which the student is enrolled.

Report	Output
Course attendance register	One student row, one column per roll call, then present, absent, and percentage totals
Teacher cohort report	One percentage column per cohort course and a formula-driven overall total
Dashboard summaries	Current totals, selected-period percentages, sessions, and department statistics
Export formats	Paginated JSON plus print-ready PDF and Excel files

Table 4.3 Implemented reporting outputs

4.3.5. Deployment and Operational Configuration

The Angular production bundle is configured as a Cloudflare Worker static asset application with single-page-application fallback routing. The production frontend connects to the HTTPS Spring API. The ESP32-S3 firmware also connects through HTTPS so that captured images and attendance results are protected in transit.

Operational settings are externalized, including the application time zone, attendance and face-similarity thresholds, current semester and academic year, scheduler interval, mail delivery, AI-service timeouts, JWT secret, database connection, and device credentials. Sensitive values are not committed to the frontend bundle.

4.4. Summary

This chapter presented the corrected hardware and software implementation of the Smart Attendance System. The physical unit uses an ESP32-S3, OV2640 camera, buzzer, green and red LEDs, PCB board, and junction box. The Angular portal supports role-specific workflows, Spring Boot coordinates security and academic rules, FastAPI performs isolated face operations, and PostgreSQL maintains authoritative records. Session discovery, student eligibility, weighted roll calls, and duplicate prevention remain server controlled across mobile, classroom-device, dashboard, and export workflows.